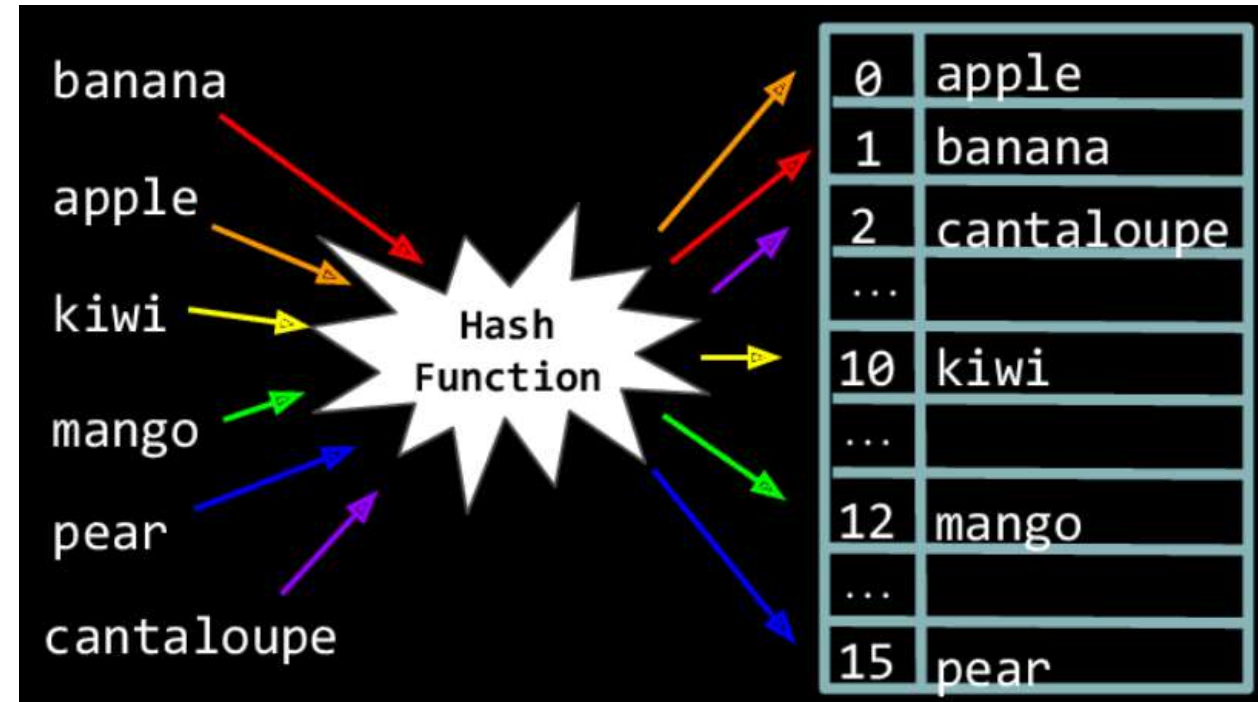


Implémentation d'un dictionnaire avec une table de hachage

rouanehm@gmail.com

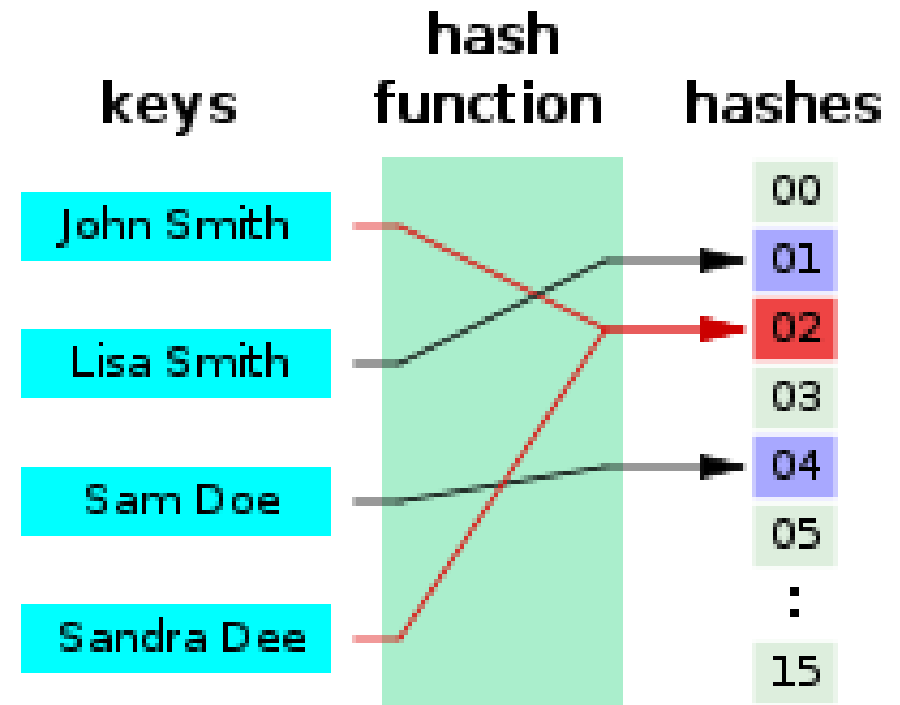
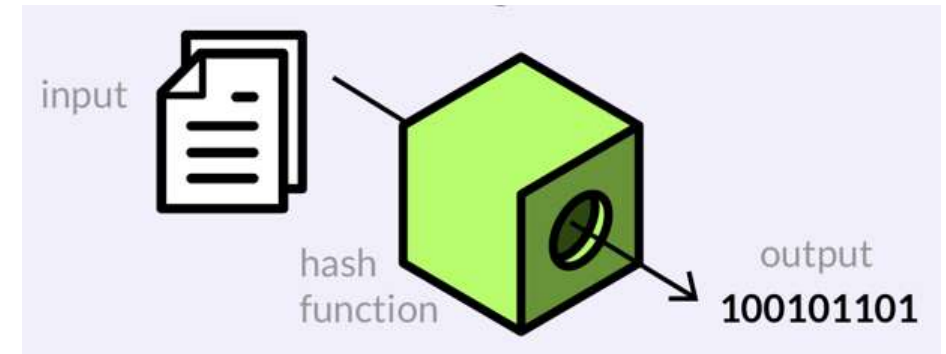
Principe d'une table de hachage:

- La table de hachage (hash table, hash map) est une structure de données qui implémente un TAD dictionnaire (tableau associatif), une structure qui (map) fait correspondre des clés à des valeurs.
- Une table de hachage utilise une fonction de hachage pour calculer un index, également appelé code de hachage, dans un tableau de compartiments ou d'emplacements (buckets or slots), à partir duquel la valeur associée à la clé peut être retrouvée.
- The values returned by a hash function are called hash values, hash codes, hash sums, or simply hashes.

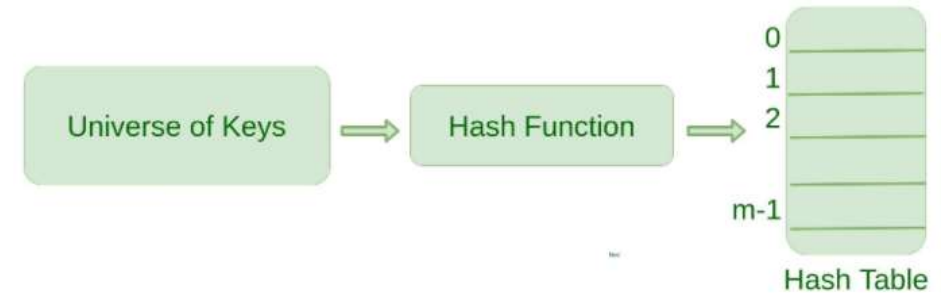


Fonction de hachage : Définition

- Inventée en 1953 par Luhn
- Idéalement, la fonction de hachage associe chaque clé à une position unique dans un tableau de stockage.
- La plupart des hash tables utilisent une fonction de hachage imparfaite, ce qui peut conduire à des collisions entre plusieurs clés qui ont un même code de hachage (les détails plus loin).



Fonction de hachage : Le principe



- Principe :
 - Utiliser une table T de taille $n \ll |U|$ (U est le domaine/univers des clés)
 - Stocker l'élément x à la position $h(x.key)$, ou h est une fonction de hachage :
- Une fonction de hachage est une fonction h qui, pour chaque élément x de l'ensemble, calcule une valeur $h(x.key)$ comprise entre 0 et $N-1$ correspondant à un indice du tableau de dimension N .

$$h : U \rightarrow \{0, \dots, m-1\}$$

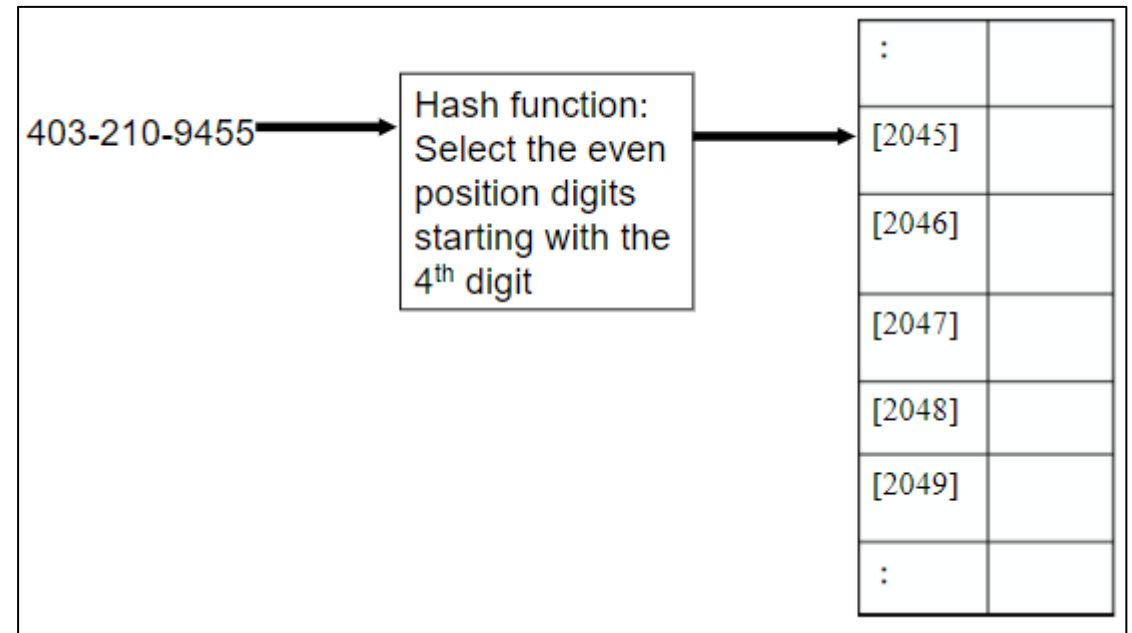
HASH-INSERT(T, x)
1 $T[h(x.key)] = x$

HASH-DELETE(T, x)
1 $T[h(x.key)] = \text{NIL}$

HASH-SEARCH(T, x)
1 **return** $T[h(x.key)]$

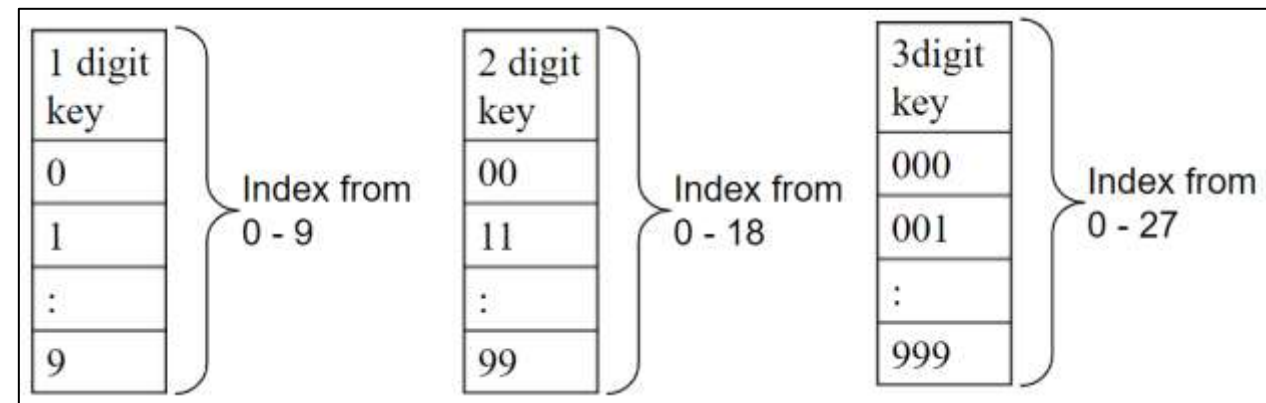
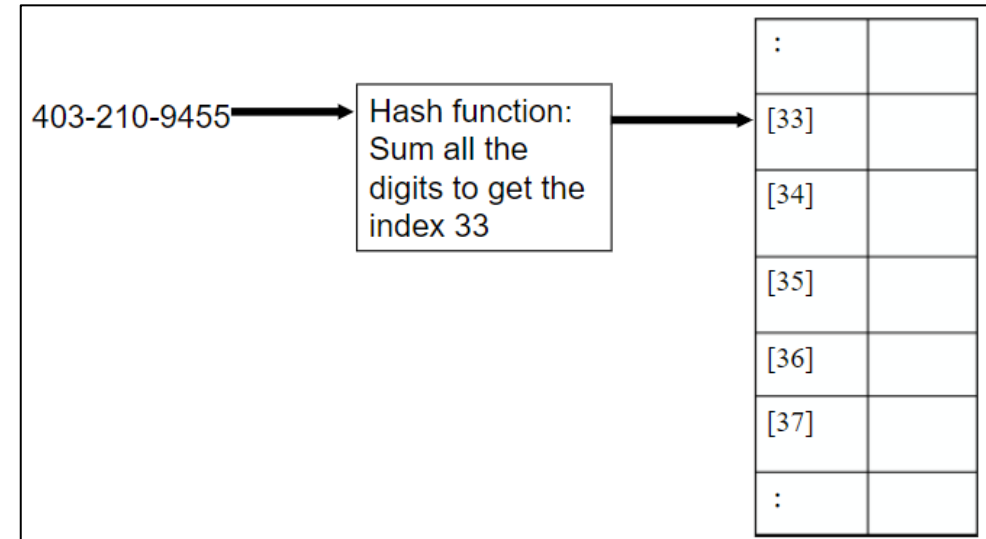
Modèles de fonctions de hachage : Selecting digits

- Select a portion of the key to use as the index into the hash table
- Works only for keys that are positive integers
- Exemple:
- Le contre :
 - The mapping of key to an index is quick
 - It usually does not evenly distribute items through the hash table (may lead to many collisions or clustering around certain parts of the hash table)



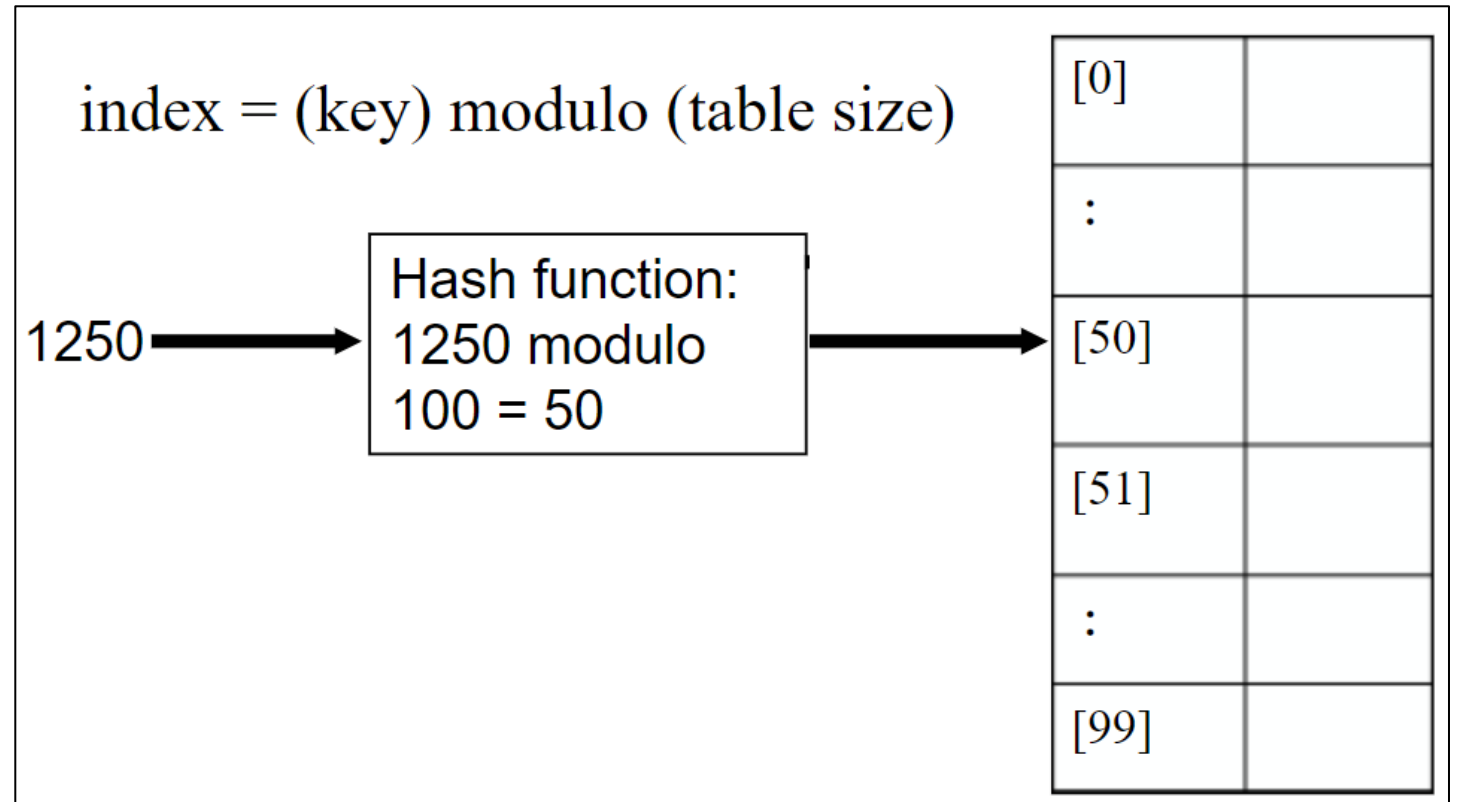
Modèles de fonctions de hachage : Folding

- Une amélioration de la méthode de sélection de chiffres car la clé est entièrement utilisée. Le domaine des clés possibles est limité.
- Une amélioration serait de faire la somme de groupe de chiffres de la clé:
 - 4032109455 --> 33
 - 4032109455 --> 4 + 032 + 109 + 455 -> 600
- Une autre amélioration serait la somme des chiffres impairs ou encore la multiplication de tous les chiffres suivie d'un décalage de bits.



Modèles de fonctions de hachage : Modular Arithmetic

- pour assurer une répartition uniforme des clés sur les différentes régions de la table de hachage, il est conseillé que la taille de cette table soit un nombre premier (ex. 101)



Fonction de hachage:

Codage des chaines de caractères

- Les fonctions de hachage supposent que les clés sont des nombres naturels. Si ce n'est pas le cas, il faut :
 - Préalablement utiliser une fonction d'encodage pour obtenir une clé
 - Utiliser un des algorithmes présentés pour trouver le hash code
- Exemple : codage des chaînes de caractères :
 - On interprète la chaîne comme un entier dans une certaine base
 - Exemple pour « SDA » : valeurs ASCII (128 possibles) :
 - S=83, D=68, A=65
 - Interprété comme un entier : $(83 \times 128^2) + (68 \times 128^1) + (65 \times 128^0) = 1368641$

```
8 def hash_code_for_string_2(key, number_of_buckets):
9     hash_code = 0
10    power = len(key) - 1
11    for letter in key:
12        hash_code = hash_code + ord(letter) * 128 ** power
13        power -= 1
14    return hash_code % number_of_buckets
```

Note: To avoid having anagrams (e.g., "NOTE" and "TONE") yielding the same integer value concatenate rather than add the results.

- TONE: 84 79 78 69 = 84797869
- NOTE: 78 79 84 69 = 78798469

Fonction de hachage:

Codage des chaines de caractères (suite)

- Une bonne fonction de hachage doit calculer rapidement une valeur ($O(1)$) mais aussi diminuer le nombre de collisions.
- Exemple: Étant donné un tableau de taille N , la fonction de hachage pour un ensemble dont les éléments sont des chaînes de caractères :

$$h(\text{chaîne}) = ((\sum \text{codes de caractères}) \bmod N)$$

chaîne	$h(\text{chaîne})$
Jean	7
Jacques	5
Vincent	6
Lucien	9
Pierre	6

```
1 def hash_code_for_string(key, number_of_buckets):
2     hash_code = 0
3     for letter in key:
4         hash_code = hash_code + ord(letter)
5     return hash_code % number_of_buckets
6
7
8 # Colval - DSA - Hashing
9 print(hash_code_for_string('Python', 10))
```

0	
1	
2	
3	
4	
5	Jacques
6	Vincent, Pierre
7	Jean
8	
9	Lucien

Fonction de hachage:

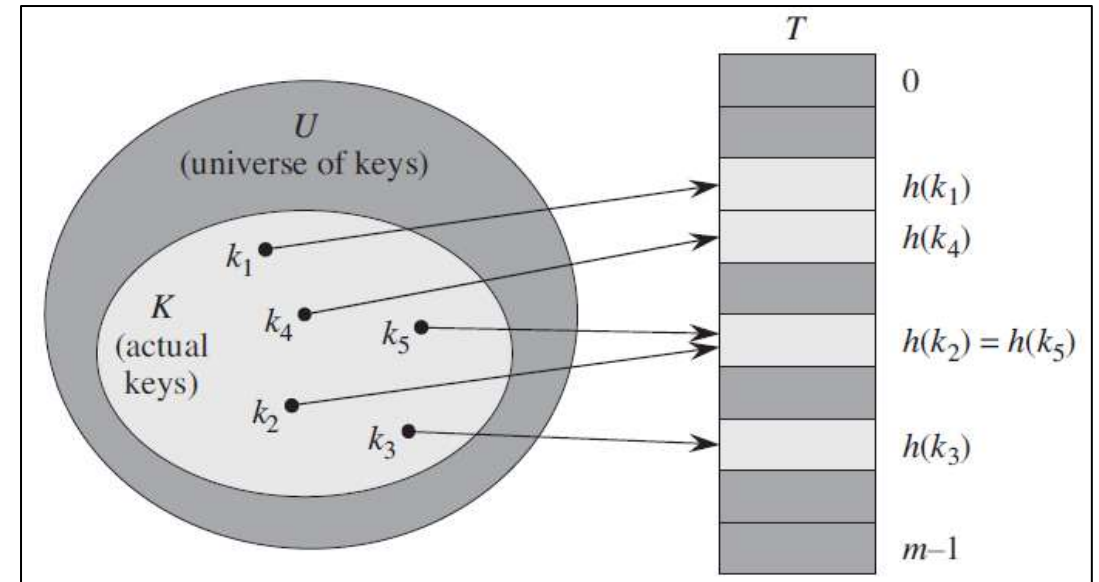
Une bonne fonction de hachage

- To achieve a good hashing mechanism, It is important to have a good hash function with the following basic requirements:
 - It should be as uncomplicated as possible and fast to compute and must not become an algorithm in itself, e.g., a single mathematical or bitwise operation.
 - Uniform distribution: It should provide a uniform distribution across the hash table and should not result in clustering.
 - Less collisions: Collisions occur when pairs of elements are mapped to the same hash value. These should be avoided.
 - Irrespective of how good a hash function is, collisions are bound to occur.
 - Typically not possible because all of the keys are not known in advance

Collision :

Définition

- **Collision** : lorsque deux clés distinctes k_1 et k_2 donne la même position dans le tableau de stockage avec la fonction de hachage h : $h(k_1) = h(k_2)$
- Cela se produit toujours lorsque le nombre de clés observées est plus grand que la taille m du tableau T :
 - $|K| > m$
- Très probable, même lorsque la fonction de hachage répartit les clés uniformément.



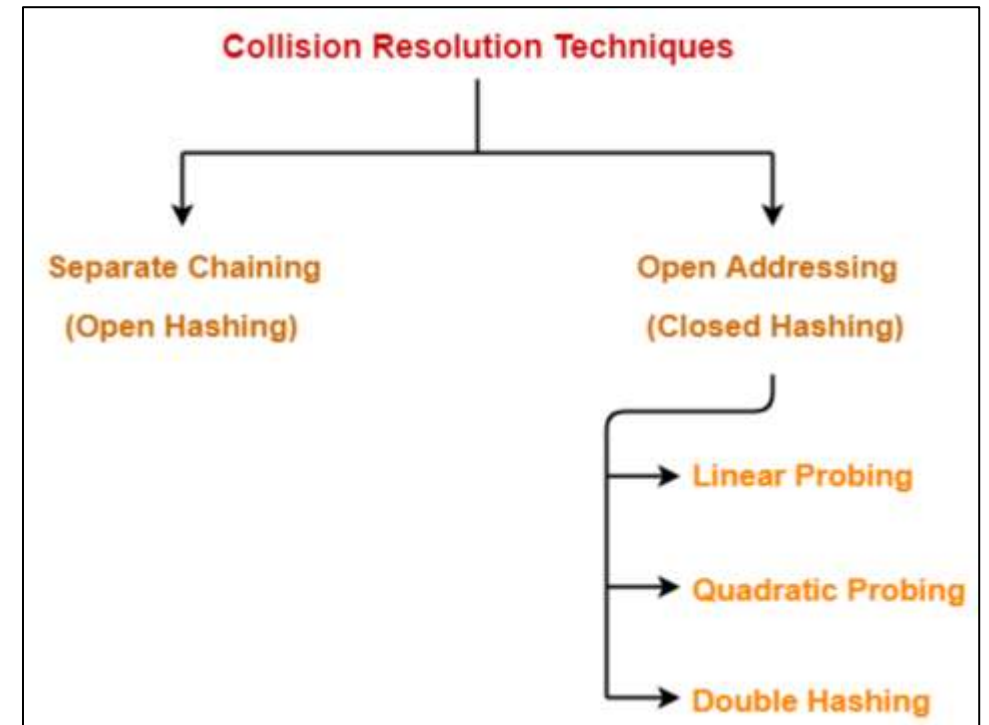
Résolution de la collision:

Deux familles d'algorithmes

- La hash table possède un tableau interne qui stocke les valeurs des clés.
- La notion d'adresse signifie la position à laquelle la valeur sera stockée.
- **Open hashing/Closed addressing/separate chaining** : Aucune valeur n'est stockée dans le tableau interne, mais plutôt dans une structure auxiliaire (appelée aussi Bucket), ex., une liste séparée (séparée du tableau interne). Le terme 'Open' fait référence à la liberté de quitter le tableau interne pour stocker les valeurs associées aux différentes clés. Le terme 'Closed' signifie qu'il est pas autorisé de replacer la valeur dans un autre endroit du tableau interne.
- **Closed hashing/Open addressing**: le tableau interne est la seule structure de stockage. Chaque valeur est stockée dans le tableau interne. Ceci est possible seulement si on autorise un certain adressage ouvert (Open addressing) qui permet de trouver, s'il y a collision, une autre place dans le tableau interne à la nouvelle valeur ajoutée.

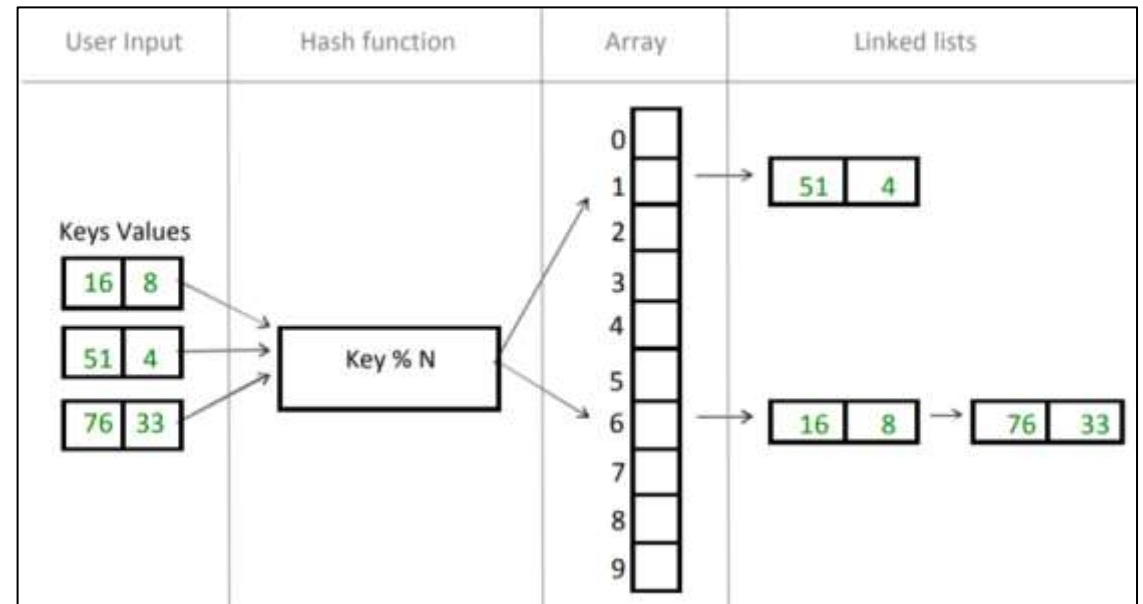
Résolution de la collision: Deux familles d'algorithmes

- Gestion par Open hashing/closed addressing
 - Separate (external) chaining
 - Buckets
- Gestion par Closed Hashing /Open Addressing
 - Linear probing
 - Quadratic probing
 - Double hashing
 - Augmentation de la dimension du tableau si taux occupation > 90%: re-hashing



Résolution de la collision: Hachage ouvert (Separate Chaining)

- En cas de collision, le tableau est «ouvert» vers la droite.
 - Ex. les éléments sont placés dans une liste (simple ou double).
- Inconvénient: plus il y a de collisions, plus les listes deviennent longues et moins la recherche est efficace.



Résolution de la collision: Code de hachage ouvert – put

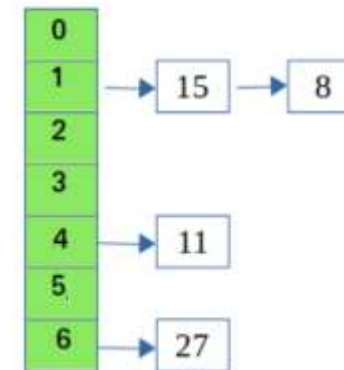
```
3 class SeparateChainingHashtable(object):
4
5     def __init__(self, capacity=1000):
6         # List of lists
7         self.storage = [[] for _ in range(capacity)]
8         self.capacity = capacity
9         self.length = 0
```

```
21     def put(self, key, value):
22         self.__setitem__(key, value)
```

```
11     def __setitem__(self, key, value):
12         hashed_key = hash(key)
13         storage_index = hashed_key % self.capacity
14         for element in self.storage[storage_index]:
15             if key == element[0]: # already exist, update it
16                 element[1] = value
17                 break
18         else:
19             # separate chaining strategy :
20             # create a chain of values in the same bucket
21             # by using another data structure (in this case a list).
22             self.storage[storage_index].append([key, value])
23             self.length += 1
```

Let's say hash table with 7 buckets (0, 1, 2, 3, 4, 5, 6)

Keys arrive in the Order (15, 11, 27, 8)



<https://www.geeksforgeeks.org/implementation-of-hashing-with-chaining-in-python/>

Résolution de la collision: Code de hachage ouvert – get/remove

```
47 def remove(self, key):
48     self.__delitem__(key)
49
50 def __delitem__(self, key):
51     index = self._find_item(key)
52     self.storage[index] = None
```

```
25 def __getitem__(self, key):
26     hashed_key = hash(key)
27     storage_index = hashed_key % self.capacity
28     for element in self.storage[storage_index]:
29         if element[0] == key:
30             return element[1]
31     raise KeyError(f'Key {key} not found')
```

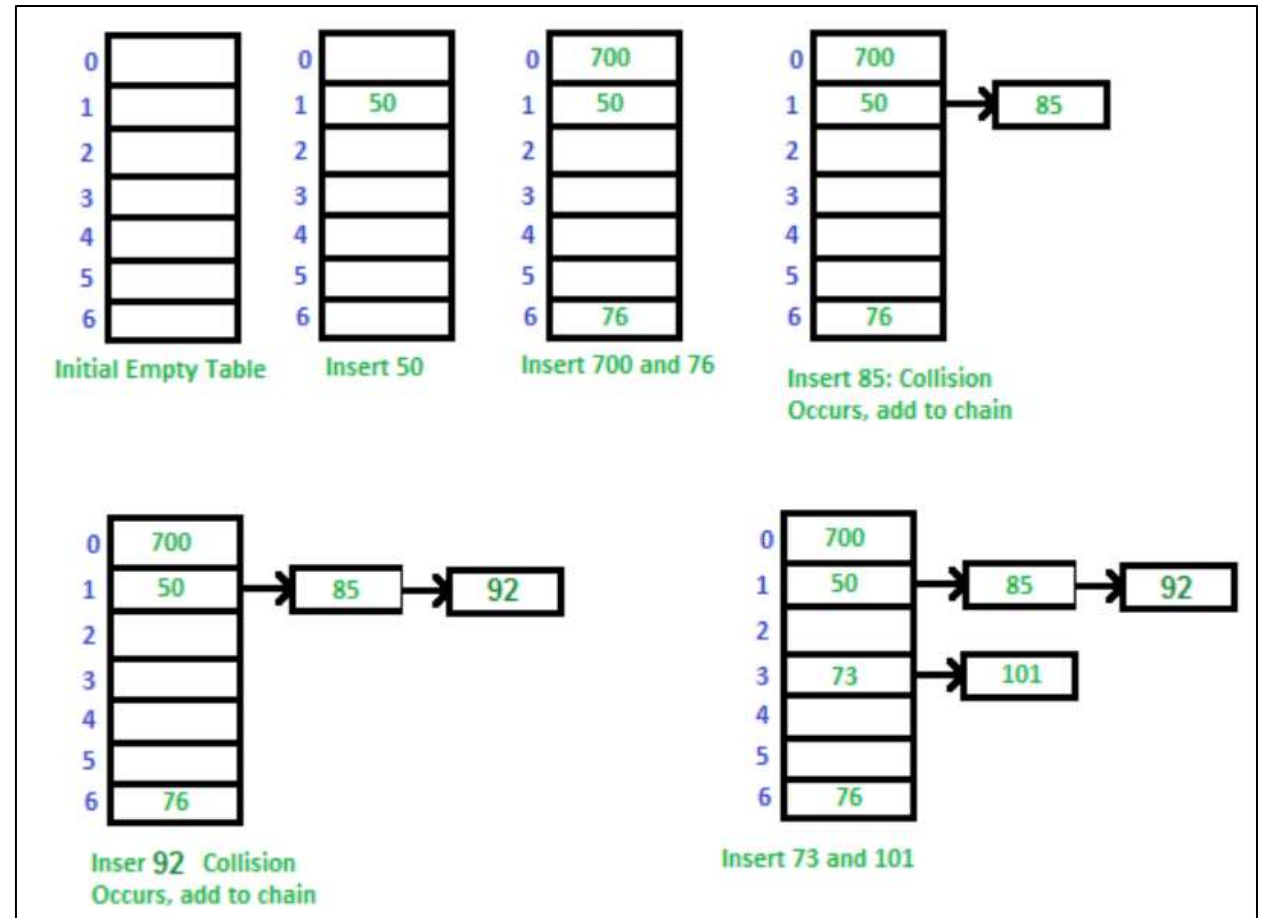
```
54 def _find_item(self, key):
55     hashed_key = self._hash(key)
56     if self.storage[hashed_key] is None:
57         raise KeyError
58     if self.storage[hashed_key][0] != key:
59         original_key = hashed_key
60         while self.storage[hashed_key][0] != key:
61             hashed_key = self._increment_key(hashed_key)
62             if self.storage[hashed_key] is None:
63                 raise KeyError
64             if hashed_key == original_key:
65                 raise KeyError
66     return hashed_key
```

```
37 def get(self, key):
38     return self.__getitem__(key)
```

Résolution de la collision: Exemple de hachage ouvert

Let us consider a simple hash function as “**key mod 7**” and sequence of keys as 50, 700, 76, 85, 92, 73, 101

<https://csrgxtu.github.io/2020/04/21/Python-Dict-a-new-implementation-by-pure-Python/>



Résolution de la collision:

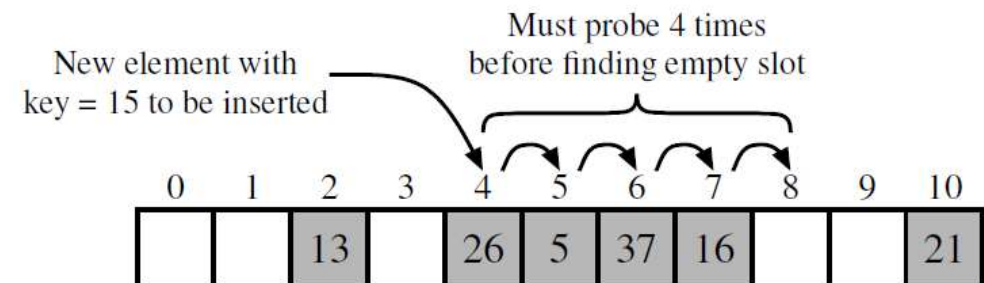
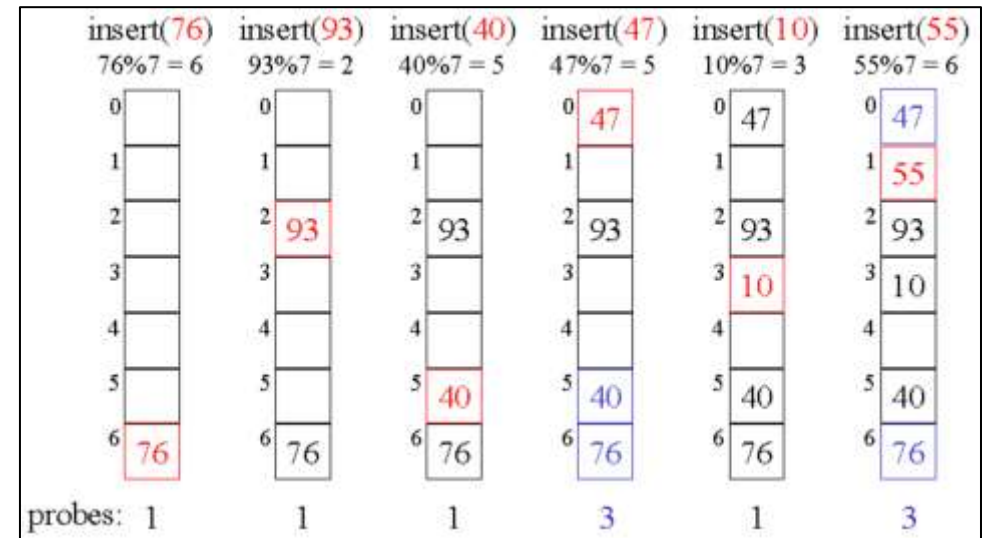
Complexité des opérations de l'hachage ouvert

- Comme il y a une liste simple pour chaque clé qui s'occupe de stocker les différentes valeurs associées à cette clé, la complexité des opérations de base (insertion, recherche et suppression) sont toutes en $O(n)$.
- On peut diminuer cette complexité si l'on choisisse une représentation moins coûteuse qu'une simple liste (liste triée, etc.)

	Worst
Space	$O(n)$
Search	$O(n)$
Insert	$O(n)$
Delete	$O(n)$

Résolution de la collision: Hachage fermé – Linear Probing

- Principe : on cherche la prochaine place libre dans le tableau pour accueillir la nouvelle valeur.
- La séquence de recherche est :
 - 1^{er} probe: $h(k) \bmod \text{table_size}$
 - 2^{ème} $h(k)+f(1) \bmod \text{table_size}$
 - 3^{ème} $h(k)+f(2) \bmod \text{table_size}$
 - Etc.
- Linear probing : $f(i) = i$
- Quadratic probing : $f(i) = i^2$
- Note: Une longue séquence de probes peut être couteuse.



Résolution de la collision: Hachage fermé – Linear Probing

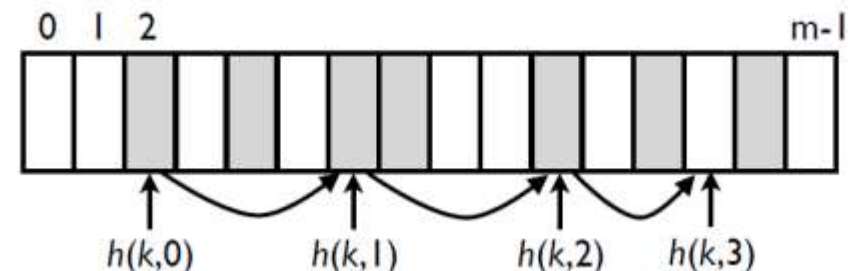
- On cherche à replacer l'élément en collision ailleurs dans le tableau en appliquant une fonction de re-hachage qui calcule un nouvel indice et ceci jusqu'à ce que la nouvelle entrée soit libre. Ce choix conduit à une dégradation des performances dès que le taux de remplissage de la table dépasse 90%.

HASH-SEARCH(T, k)

```
1   $i = 0$ 
2  repeat
3       $j = h(k, i)$ 
4      if  $T[j] == k$ 
5          return  $j$ 
6       $i = i + 1$ 
7  until  $T[j] == \text{NIL}$  or  $i == m$ 
8  return NIL
```

HASH-INSERT(T, k)

```
1   $i = 0$ 
2  repeat
3       $j = h(k, i)$ 
4      if  $T[j] == \text{NIL}$ 
5           $T[j] = k$ 
6          return  $j$ 
7      else  $i = i + 1$ 
8  until  $i == m$ 
9  error "hash table overflow"
```



Résolution de la collision: Code de hachage fermé Linear Probing – put

```
3 class LinearProbingHashTable(object):
4
5     def __init__(self):
6         self.max_length = 10
7         self.max_load_factor = 0.75
8         self.length = 0
9         self.storage = [None for _ in range(self.max_length)]
10        # self.storage = [None] * self.max_length
```

```
33     def put(self, key, value):
34         self.__setitem__(key, value)
```

```
14     def _hash(self, key):
15         return hash(key) % self.max_length
16
17     def _increment_key(self, key): # Linear probing
18         return (key + 1) % self.max_length
19
20     def __setitem__(self, key, value):
21         self.length += 1
22         hashed_key = self._hash(key)
23         while self.storage[hashed_key] is not None:
24             if self.storage[hashed_key][0] == key:
25                 self.length -= 1
26                 break
27             hashed_key = self._increment_key(hashed_key)
28         new_tuple = (key, value)
29         self.storage[hashed_key] = new_tuple
30         if self.length / float(self.max_length) >= self.max_load_factor:
31             self._resize()
```

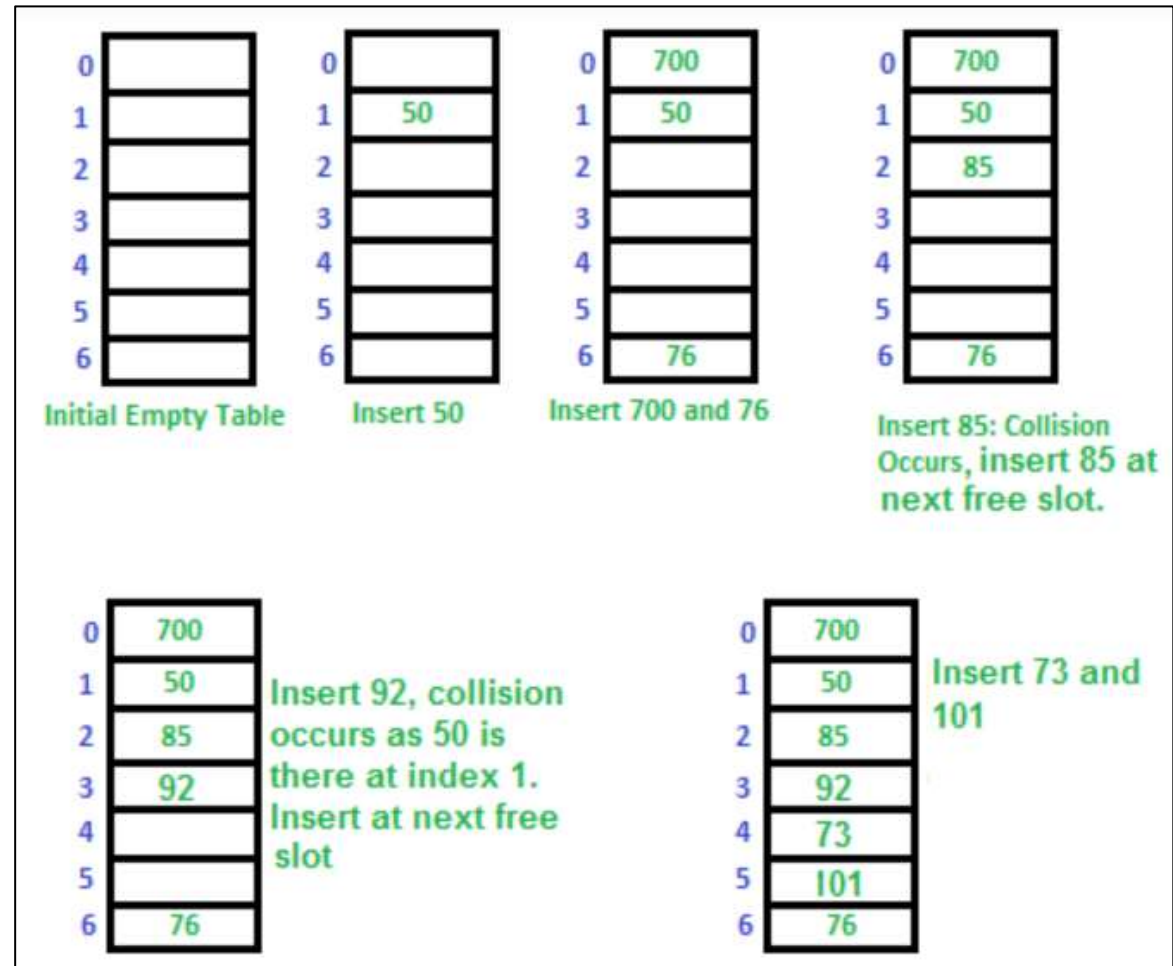
Résolution de la collision: Code de hachage fermé Linear Probing – get/remove

```
50 def _find_item(self, key):
51     hashed_key = self._hash(key)
52     if self.storage[hashed_key] is None:
53         raise KeyError
54     if self.storage[hashed_key][0] != key:
55         original_key = hashed_key
56         while self.storage[hashed_key][0] != key:
57             hashed_key = self._increment_key(hashed_key)
58             if self.storage[hashed_key] is None:
59                 raise KeyError
60             if hashed_key == original_key:
61                 raise KeyError
62     return hashed_key
```

```
36 def get(self, key):
37     return self.__getitem__()
38
39 def __getitem__(self, key):
40     index = self._find_item(key)
41     return self.storage[index][1]
42
43 def remove(self, key):
44     self.__delitem__()
45
46 def __delitem__(self, key):
47     index = self._find_item(key)
48     self.storage[index] = None
```

Résolution de la collision: Exemple de hachage fermé

Let us consider a simple hash function as “key mod 7” and sequence of keys as 50, 700, 76, 85, 92, 73, 101.



Résolution de la collision:

Complexité des opérations de l'hachage fermé

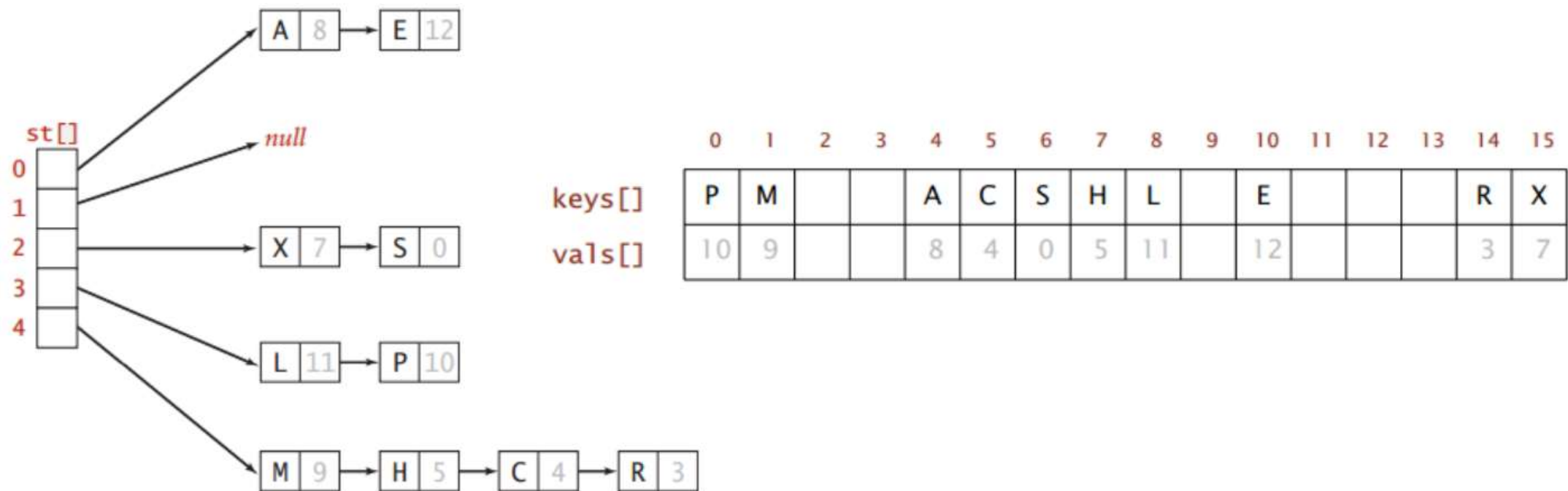
Cas Linear Probing

- Toutes les opérations sont en $O(n)$ à cause de la recherche de l'emplacement de la clé à l'intérieur du tableau interne de la table de hachage.

Algorithm	Worst case
Access	$O(n)$
Search (by key)	$O(n)$
Search (by value)	$O(n)$
Insert	$O(n)$
Delete	$O(n)$
-	-
Space	$O(n)$

Adressage ouvert versus adressage fermé :

Exemple récapitulatif



Adressage ouvert versus adressage fermé : le pour et le contre

- Adressage fermé (chainage)
 - Peut gérer un nombre illimité d'éléments et de collisions
 - Performances plus stables
 - Surcoût lié à la gestion et le stockage en mémoire des listes liées
- Adressage ouvert
 - Rapide et peu gourmand en mémoire
 - Choix de la fonction de hachage plus difficile (pour éviter les grappes)
 - On ne peut pas avoir $n > m$
 - Suppression problématique
- D'autres alternatives existent :
 - Two-probe hashing
 - Cuckoo hashing

	Closed hashing (open addressing)	Open hashing: separate chaining)
Description	•Resolve collisions by finding another place in the hash table	•Resolve collisions by inserting additional elements at the same location in the hash table
Strengths	•May be faster in practice because the table doesn't change in size	•May require less memory (smaller hash table) •Fewer compares
Weaknesses	•Requires a larger hash table (percentage utilization of the table should be lower than with open hashing)	•May be slower in practice because of the dynamic memory allocations •More complex: needs another data structure

Complexité des opérations d'un dictionnaire versus les autres implémentations de TAD

implementation			
	search	insert	delete
sequential search (unordered list)	N	N	N
binary search (ordered array)	$\lg N$	N	N
BST	N	N	N
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$
separate chaining	N	N	N
linear probing	N	N	N

Data Structure	Time Complexity				Space Complexity
	Worst				Worst
	Access	Search	Insertion	Deletion	
<u>Array</u>	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Stack</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Queue</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Singly-Linked List</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Doubly-Linked List</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Skip List</u>	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n \log(n))$
<u>Hash Table</u>	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Binary Search Tree</u>	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Cartesian Tree</u>	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>B-Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>Red-Black Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>Splay Tree</u>	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>AVL Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>KD Tree</u>	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

<https://www.bigocheatsheet.com/>

Références

- <https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/Dictionary.html>
- <http://sdz.tdct.org/sdz/les-tables-de-hachage-1.html>
- <https://slideplayer.com/slide/4793596/>
- <https://www.interviewcake.com/concept/java/hash-map>
- <https://medium.com/omarelgabrys-blog/hash-tables-2fec6870207f>